

Retrieval & RAG

A Complete Interview Preparation Guide

Theory · Mathematics · Implementation Deep-Dive

About this document

This guide covers the full theory of information retrieval and retrieval-augmented generation (RAG) — from classical BM25 to dense bi-encoders, ColBERT, reranking, and production RAG pipelines — culminating in a deep-dive into the **hindyros/RAG** system, including every architectural decision and its justification.

Use case: Panel interview preparation. Every section is designed so that you can explain the “why” behind each decision, not just the “what.”

Contents

1	Why Retrieval?	3
1.1	The Knowledge Bottleneck	3
1.2	What Retrieval Solves	3
1.3	The General Retrieval Pipeline	3
2	Sparse Retrieval	3
2.1	TF-IDF	3
2.1.1	Term Frequency	3
2.1.2	Inverse Document Frequency	3
2.1.3	TF-IDF Score	4
2.2	BM25 (Best Match 25)	4
2.2.1	Key Improvement 1: TF Saturation	4
2.2.2	Key Improvement 2: Length Normalization	4
3	Dense Retrieval	4
3.1	Bi-Encoder Architecture	5
3.2	Dense Passage Retrieval (DPR)	5
3.2.1	Training Objective	5
3.2.2	In-Batch Negatives	5
3.2.3	Hard Negatives	5
3.3	Contrastive Learning Objective	5
3.4	Bi-Encoder Limitations	6
4	Late Interaction: ColBERT	6
4.1	Token-Level Representations	6
4.2	MaxSim Operator	6
4.3	Why “Late” Interaction?	6
4.4	ColBERTv2 Improvements	7
5	Cross-Encoders and Neural Re-Ranking	7
5.1	Cross-Encoder Architecture	7
5.2	Two-Stage Retrieve-then-Rerank Pipeline	7
5.3	Knowledge Distillation from Re-Ranker to Retriever	7
6	Retrieval-Augmented Generation (RAG)	8
6.1	The RAG Framework	8
6.2	RAG-Sequence vs. RAG-Token	8
6.3	RAG Training Approaches	8
6.4	Lost in the Middle	8
7	Evaluation of Retrieval Systems	8
7.1	Recall@k	8
7.2	Mean Reciprocal Rank (MRR)	9
7.3	Mean Average Precision (MAP)	9
7.4	NDCG (Normalized Discounted Cumulative Gain)	9
7.5	End-to-End QA Metrics	9

8 Multi-Hop Retrieval	9
8.1 The Multi-Hop Problem	9
8.2 Iterative Retrieval	10
8.3 IRCoT — Interleaved Chain-of-Thought Retrieval	10
8.4 FLARE — Forward-Looking Active Retrieval	10
9 Sparse-Dense Hybrid Retrieval	10
9.1 Why Hybrid?	10
9.2 Linear Combination	10
9.3 Reciprocal Rank Fusion (RRF)	11
9.4 SPLADE	11
10 HyDE: Hypothetical Document Embeddings	11
11 The BEIR Benchmark	11
12 Summary: The Retrieval Hierarchy	12
My RAG System: hindyros/RAG	13
File Structure	13
Ingestion Pipeline	13
Query Pipeline	15
Query Pipeline — Step by Step	15
Architectural Decisions	18
Configuration Reference	20
Embedding Models	21
API Endpoints	21
Performance Characteristics	21
Interview Talking Points	21

Why Retrieval?

The Knowledge Bottleneck

A language model with N parameters stores knowledge *implicitly* in its weights. There is a fundamental tension between two kinds of memory:

Type	Strengths	Weaknesses
Parametric (weights)	Fast (single forward pass)	Static, opaque, hallucination-prone
Non-parametric (external DB)	Dynamic, auditable, arbitrarily large	Requires retrieval mechanism

Retrieval-augmented models combine both: **parametric knowledge for reasoning and language, non-parametric knowledge for specific facts.**

What Retrieval Solves

Problem	How Retrieval Helps
Knowledge cutoff	Retrieve from an up-to-date corpus
Hallucination	Ground generation in retrieved evidence
Attribution	Show the user which documents support the answer
Long-tail facts	Retrieve rare facts rather than hope they were memorized
Specialized domains	Index domain-specific corpora (medical, legal, scientific)

The General Retrieval Pipeline

```

Query q
|
[Retriever] --> top-k documents {d1, ..., dk}
|
[Reader / Generator] (LM conditioned on q and {d1,...,dk})
|
Answer / Generation

```

The retriever and generator can be trained jointly or independently. Most research focuses on making the retriever fast, accurate, and trainable end-to-end.

Sparse Retrieval

Sparse methods represent queries and documents as high-dimensional *sparse* vectors — most entries are zero. Matching is based on term overlap.

TF-IDF

Term Frequency

$$\text{TF}(t, d) = \frac{f_{t,d}}{\sum_{t'} f_{t',d}}$$

where $f_{t,d}$ is the raw count of term t in document d .

Inverse Document Frequency

$$\text{IDF}(t) = \log \frac{N}{|\{d : t \in d\}|}$$

where N is the total number of documents.

TF-IDF Score

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \cdot \text{IDF}(t)$$

Intuition: TF rewards words that appear frequently *in this document*. IDF penalizes words that appear in almost *every* document (“the”, “is”). Together, TF-IDF weights words that are common in this document but rare globally.

Document retrieval: Represent query q and document d as TF-IDF vectors in $\mathbb{R}^{|V|}$ and rank by cosine similarity:

$$\text{score}(q, d) = \frac{\mathbf{v}_q \cdot \mathbf{v}_d}{\|\mathbf{v}_q\| \|\mathbf{v}_d\|}$$

TF-IDF Limitations

- **Vocabulary mismatch:** “automobile” vs. “car” yields zero overlap.
- **No semantics:** “I love Paris” and “I hate Paris” are identical vectors.
- **Curse of dimensionality:** $|V|$ can exceed 500K.

BM25 (Best Match 25)

BM25 is the dominant sparse retrieval algorithm — used in Elasticsearch, Lucene, and virtually all production search systems. It improves on TF-IDF with two key modifications.

BM25 Score

$$\text{BM25}(q, d) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f_{t,d} \cdot (k_1 + 1)}{f_{t,d} + k_1 \left(1 - b + b \cdot \frac{|d|}{\text{avgdl}} \right)}$$

Where: $f_{t,d}$ = term count; $|d|$ = doc length; avgdl = average doc length; $k_1 \in [1.2, 2.0]$ (TF saturation); $b \in [0, 1]$ (length normalization, typically 0.75).

Key Improvement 1: TF Saturation

TF-IDF grows *linearly* with term frequency. BM25 *saturates*:

$$\lim_{f_{t,d} \rightarrow \infty} \frac{f_{t,d} \cdot (k_1 + 1)}{f_{t,d} + k_1} = k_1 + 1$$

A term appearing 1000 times scores almost the same as one appearing 10 times. k_1 controls how quickly saturation kicks in.

Key Improvement 2: Length Normalization

Long documents contain more terms by chance. BM25 adjusts: a term appearing once in a 5-word document is more informative than once in a 5000-word document. $b = 0$: no normalization; $b = 1$: full normalization.

Why BM25 is Hard to Beat

Despite its simplicity, BM25 is a remarkably strong baseline. On many benchmarks, a well-tuned BM25 outperforms dense neural retrievers trained on different distributions (see BEIR, Section 11). It remains the default in most production systems.

Dense Retrieval

Dense retrieval maps queries and documents to dense vectors in $\mathbb{R}^d$ (typically $d = 768$ or $d = 1024$). Relevance is measured by vector similarity, not term overlap.

Bi-Encoder Architecture

The **bi-encoder** (dual encoder) encodes query and document *independently*:

$$\mathbf{q} = E_Q(q) \in \mathbb{R}^d, \quad \mathbf{d} = E_D(d) \in \mathbb{R}^d$$

$$\text{sim}(q, d) = \mathbf{q}^\top \mathbf{d}$$

E_Q and E_D may share weights or be separate networks (e.g., BERT with [CLS] pooling).

Critical Property: Pre-computation

Because query and document encodings are *independent*, all document embeddings can be pre-computed offline and stored in an index. At query time, only $E_Q(q)$ needs to be computed, then we perform nearest-neighbor search over the document index. This makes bi-encoders fast enough for first-stage retrieval over millions of documents.

Approximate Nearest Neighbor (ANN): Exact nearest-neighbor in $\mathbb{R}^d$ over millions of documents is infeasible. FAISS, ScaNN, and HNSW enable sub-linear search with small recall penalties.

Dense Passage Retrieval (DPR)

DPR (Karpukhin et al., 2020) is the seminal dense retrieval model for open-domain QA.

- E_Q : BERT-base, takes question as input
- E_D : BERT-base (separate weights), takes passage as input
- Embedding: [CLS] token representation

Training Objective

Given question q with positive passage d^+ and m hard negatives $\{d_j^-\}$:

$$\mathcal{L}_{\text{DPR}} = -\log \frac{e^{\mathbf{q}^\top \mathbf{d}^+}}{e^{\mathbf{q}^\top \mathbf{d}^+} + \sum_{j=1}^m e^{\mathbf{q}^\top \mathbf{d}_j^-}}$$

Softmax cross-entropy loss pushing q close to d^+ and away from d_j^- .

In-Batch Negatives

Within a training batch of B (question, positive passage) pairs, each question's negatives are the positive passages of all other questions — giving $B - 1$ negatives per example for free, with no additional forward passes.

Hard Negatives

- **Easy negative:** Passage about cooking when question is about US history.
- **Hard negative:** Passage about War of 1812 when question asks about the date of the Battle of Waterloo.

BM25-retrieved passages that look lexically relevant but are semantically wrong make the best hard negatives. DPR trains with both in-batch and BM25 hard negatives.

Contrastive Learning Objective

DPR's loss is an instance of **contrastive learning**: pull together (query, relevant doc) pairs, push apart (query, irrelevant doc) pairs.

The InfoNCE loss generalizes this with a temperature parameter τ :

$$\mathcal{L}_{\text{InfoNCE}} = -\frac{1}{B} \sum_{i=1}^B \log \frac{e^{\text{sim}(q_i, d_i^+)/\tau}}{\sum_{j=1}^B e^{\text{sim}(q_i, d_j)/\tau}}$$

Temperature intuition: Small $\tau \rightarrow$ sharp, confident distributions (risky with noisy labels); large $\tau \rightarrow$ soft, gentle gradients (better for ambiguous data).

Bi-Encoder Limitations

The **independence constraint** is the fundamental limitation. Query and document embeddings are computed separately — there is no cross-attention between them. This means the model cannot condition the document representation on specific query terms. Complex queries requiring fine-grained matching can fool a coarse bi-encoder.

This motivates re-rankers and late-interaction models.

Late Interaction: ColBERT

ColBERT (Contextualized Late Interaction over BERT, Khattab & Zaharia, 2020) is a middle ground between the efficiency of bi-encoders and the expressiveness of cross-encoders.

Token-Level Representations

Unlike DPR, which pools to a single vector, ColBERT keeps *token-level* representations:

$$\mathbf{Q} = \text{normalize}(\text{BERT}_Q(q)) \in \mathbb{R}^{|q| \times d}$$

$$\mathbf{D} = \text{normalize}(\text{BERT}_D(d)) \in \mathbb{R}^{|d| \times d}$$

MaxSim Operator

ColBERT MaxSim

$$\text{score}(q, d) = \sum_{i=1}^{|q|} \max_{j=1}^{|d|} \mathbf{Q}_i \cdot \mathbf{D}_j^\top$$

For each query token i , find the document token j^* that is most similar to it, take that similarity, and sum across all query tokens.

Intuition: Each query token “finds its evidence” in the document. The score is high when every query concept has a corresponding mention in the document. This is **soft matching**: query token “automobile” can match document token “car” via embedding proximity — unlike BM25’s hard lexical matching.

Why “Late” Interaction?

ColBERT uses **late interaction** — encoders run independently (like bi-encoders), but relevance is computed via fine-grained token-level interaction (unlike bi-encoders):

```
Offline: BERT_D(d) --> store all token embeddings in index
Online: BERT_Q(q) --> query token embeddings --> MaxSim over index
```

Key efficiency gain: document token embeddings are pre-computed. At query time, only the query encoder runs + token-level max similarities. More expensive than bi-encoder ANN search, but far cheaper than cross-encoder inference.

ColBERTv2 Improvements

1. **Residual compression:** Token embeddings compressed via vector quantization (codebook + residual over centroid), dramatically reducing storage.
2. **Cross-encoder distillation:** A cross-encoder teacher generates soft labels that supervise ColBERT's training, transferring richer relevance signals.

Method	Storage (MS MARCO)	Latency	MRR@10
BM25	~2 GB (inverted index)	~20ms	18.4
DPR	~26 GB (dense index)	~5ms (ANN)	31.7
ColBERT	~154 GB (token embeddings)	~100ms	36.0
ColBERTv2	~25 GB (compressed)	~100ms	39.7
Cross-encoder	N/A (cannot pre-index)	~1s per doc	42+

Cross-Encoders and Neural Re-Ranking

Cross-Encoder Architecture

A **cross-encoder** concatenates query and document as a single input:

$$\text{score}(q, d) = \text{MLP}(\text{BERT}([q; [\text{SEP}]; d]))$$

Attention layers can attend across both query and document simultaneously — every query token attends to every document token. This produces far more expressive relevance scores.

Fatal Limitation

Because query and document must be concatenated, documents *cannot be pre-indexed*. Scoring N documents for a new query requires N full forward passes. Too slow for first-stage retrieval over millions of documents.

Two-Stage Retrieve-then-Rerank Pipeline

```

Query q
|
Stage 1 (Retriever): BM25 or bi-encoder
--> top-k1 candidates (k1 ~ 100-1000), FAST
|
Stage 2 (Re-ranker): Cross-encoder
--> top-k2 re-ranked (k2 ~ 10-20), ACCURATE
|
Final ranked list

```

The retriever casts a wide net cheaply; the re-ranker scores the shortlist accurately.

Re-ranker training loss (binary cross-entropy on positive/negative pairs):

$$\mathcal{L}_{\text{rerank}} = - \sum_{(q, d^+, d^-)} \left[\log \sigma(s(q, d^+)) + \log(1 - \sigma(s(q, d^-))) \right]$$

Knowledge Distillation from Re-Ranker to Retriever

The cross-encoder computes more accurate relevance scores than the bi-encoder. **Distillation** transfers this knowledge:

$$\mathcal{L}_{\text{distill}} = \sum_{(q, d)} (s_{\text{student}}(q, d) - s_{\text{teacher}}(q, d))^2$$

This is the core technique in ColBERTv2 and SPLADE.

Retrieval-Augmented Generation (RAG)

The RAG Framework

RAG (Lewis et al., 2020) integrates retrieval directly into generation. The model generates answers conditioned on retrieved documents:

$$p(y | q) = \sum_{d \in \text{top-}k} p(d | q) \cdot p(y | q, d)$$

This is a **mixture model**: the final answer is a weighted combination over possible retrieved documents.

RAG-Sequence vs. RAG-Token

RAG-Sequence — use the same document for the entire generated sequence:

$$p_{\text{seq}}(y | q) = \sum_d p(d | q) \prod_{t=1}^{|y|} p(y_t | q, d, y_{1:t-1})$$

RAG-Token — at each token position, the model can draw from a different document:

$$p_{\text{tok}}(y | q) = \prod_{t=1}^{|y|} \sum_d p(d | q) \cdot p(y_t | q, d, y_{1:t-1})$$

In practice, most production RAG systems use the simpler **concatenation-based** approximation:

$$p(y | q) \approx p_{\theta}(y | q, d_1, d_2, \dots, d_k)$$

All top- k documents are concatenated into context; the LLM attends to all at once.

RAG Training Approaches

- **Frozen retriever + trained generator**: Simplest. BM25/DPR fixed; only LM trained. Stable, often sufficient.
- **Joint training (REALM/ATLAS)**: Both retriever and generator trained end-to-end via EM. The E-step fetches documents; the M-step trains both. Harder to implement but higher ceiling.

Lost in the Middle

A critical empirical finding (Liu et al., 2023): LLMs are significantly worse at using information that appears **in the middle** of a long context. Performance follows a U-shaped curve — high accuracy at start/end, low in the middle.

RAG Implication

Position the most relevant document first or last in context. Do not simply concatenate in retrieval-score order. Use re-ranking and put the best document at the top.

Evaluation of Retrieval Systems

Recall@k

$$\text{Recall@}k = \frac{|\text{relevant} \cap \text{retrieved}_k|}{|\text{retrieved}_k|}$$

For a single relevant document: $\text{Recall@}k \in \{0, 1\}$.

Use when: You care about whether the answer is retrievable at all. Common for open-domain QA where the generator only succeeds if the answer passage is retrieved.

Mean Reciprocal Rank (MRR)

$$\text{MRR} = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{\text{rank}_i}$$

Rank of relevant doc	Reciprocal Rank
1st	1.000
2nd	0.500
3rd	0.333
10th	0.100
Not in top-k	0.000

MRR rewards finding the relevant document earlier in the ranking. Standard in QA benchmarks (NQ, TriviaQA) with a single correct answer.

Mean Average Precision (MAP)

For queries with *multiple* relevant documents:

$$\text{AP}(q) = \frac{1}{|\text{relevant}|} \sum_{k: d_k \text{ is relevant}} \text{Precision@}k$$

$$\text{MAP} = \frac{1}{|Q|} \sum_q \text{AP}(q)$$

NDCG (Normalized Discounted Cumulative Gain)

NDCG handles **graded relevance** — not all relevant documents are equally good.

$$\text{DCG@}k = \sum_{i=1}^k \frac{r_i}{\log_2(i+1)} \quad \text{NDCG@}k = \frac{\text{DCG@}k}{\text{IDCG@}k}$$

where $r_i \in \{0, 1, 2, 3\}$ is graded relevance, and IDCG is the ideal (perfect) ranking's DCG. The discount $\frac{1}{\log_2(i+1)}$ encodes the assumption that users read top to bottom: position 1 contributes fully; position 10 contributes only ≈ 0.29 .

End-to-End QA Metrics

For RAG systems, evaluate the full pipeline on downstream QA accuracy:

- **Exact Match (EM):** Does the generated answer exactly match gold?
- **F1:** Token-level overlap between generated and gold answer.
- **LLM judge:** For open-ended questions, use GPT-4 to assess quality.

Important insight

Retrieval Recall@k does *not* perfectly predict downstream QA accuracy. A model with worse retrieval may still answer correctly from partial context. Always evaluate the full pipeline end-to-end.

Multi-Hop Retrieval

The Multi-Hop Problem

Many questions require synthesizing information from *multiple* documents:

“Who is the CEO of the company that acquired DeepMind?”

Requires: (1) Retrieve: “Google acquired DeepMind in 2014.” (2) Retrieve: “Sundar Pichai is the CEO of Google.” (3) Combine → “Sundar Pichai.”

Iterative Retrieval

```
Query q0 = original question
For step t = 1, 2, ..., T:
    Retrieve dt using current query q(t-1)
    Update context: ct = c(t-1) union {dt}
    Generate follow-up query: qt = LM(q0, ct)
Final answer = LM(q0, cT)
```

The query at each step is reformulated based on what has been retrieved so far.

IRCoT — Interleaved Chain-of-Thought Retrieval

At each step, the model:

1. Generates the next reasoning step (chain-of-thought sentence)
2. Uses that sentence as the retrieval query for the next document
3. Continues until generating a final answer

Chain-of-thought naturally decomposes the question; each sub-question becomes an effective retrieval query.

FLARE — Forward-Looking Active Retrieval

FLARE triggers retrieval *proactively when the model is uncertain*:

1. Generate tokens; when some tokens have low probability, pause.
2. Retrieve documents relevant to the low-confidence span.
3. Regenerate the uncertain span with retrieved context.

Avoids retrieval at every step; focuses effort where the model is uncertain.

Sparse-Dense Hybrid Retrieval

Why Hybrid?

Neither BM25 nor dense retrieval dominates on all query types:

BM25 is better for	Dense is better for
Exact matches, named entities	Semantic similarity, synonyms
Rare keywords (“COVID-19 RdRp inhibitor assay”)	Paraphrases (“automobile” ↔ “car”)
Out-of-distribution domains	Conceptual questions

Linear Combination

$$\text{score}_{\text{hybrid}}(q, d) = \alpha \cdot \text{BM25}(q, d) + (1 - \alpha) \cdot \text{dense}(q, d)$$

$\alpha \approx 0.3$ – 0.5 often outperforms either method alone on standard benchmarks.

Reciprocal Rank Fusion (RRF)

RRF Formula

$$\text{RRF-score}(d, R_1, R_2) = \frac{1}{K + \text{rank}_{R_1}(d) + 1} + \frac{1}{K + \text{rank}_{R_2}(d) + 1}$$

Typically $K = 60$. Documents are re-ranked by summing their RRF scores across all retrieval lists.

Advantages of RRF over linear combination:

- Rank-based: invariant to the absolute magnitude of scores from each system.
- Scale-free: no parameter tuning for relative score normalization.
- Robust: consistently outperforms linear combination in empirical studies by 3–8 points.

SPLADE

SPLADE (Formal et al., 2021) learns a *sparse* neural representation — a dense model that outputs a sparse vector in vocabulary space, enabling inverted-index retrieval with learned term expansion:

$$\mathbf{v}_d = \text{ReLU}(\log(1 + \text{BERT}(d) \cdot W))$$

SPLADE can expand terms: if the document says “cardiac arrest,” SPLADE may assign weight to “heart attack,” bridging vocabulary without full dense retrieval compute cost.

HyDE: Hypothetical Document Embeddings

HyDE (Gao et al., 2022) bridges the distributional gap between short queries and long documents:

1. **Generate:** LLM creates a plausible document that would answer the query (temperature 0.7, max 300 tokens).
2. **Embed:** The generated passage is embedded with the same embedding model as the corpus.
3. **Retrieve:** The *hypothetical document embedding* is used for dense search; the original query is still used for BM25.

Why HyDE Works

Queries are short and terse (e.g., “when did X happen?”). Documents are long and descriptive (“X occurred on Y date due to Z...”). These live in different parts of the embedding space. HyDE creates a document-like vector from the query side, unifying the search signal.

Cost: one extra LLM call per query. Worth it on knowledge-dense corpora.

The BEIR Benchmark

BEIR (Thakur et al., 2021) is the standard benchmark for **zero-shot retrieval generalization** across 18 diverse datasets:

Domain	Dataset	Task
Biomedical	BioASQ, TREC-COVID	QA, fact verification
Scientific	SCIFACT, NFCorpus	Claim verification, QA
News	TREC-NEWS, Robust04	Article retrieval
Entity	DBPedia, FEVER	Entity retrieval, fact verification
General	NQ, HotpotQA, MS MARCO	Open-domain QA

Key BEIR Finding

Dense models trained on MS MARCO often perform *worse* than BM25 on out-of-domain datasets. Retrieval models overfit to their training distribution. Zero-shot retrieval generalization remains an open research problem.

Summary: The Retrieval Hierarchy

Method	Representation	Matching	Pre-index?	Quality
BM25	Sparse bag-of-words	Lexical overlap	Yes	Low
Bi-encoder (DPR)	Single dense vector	Dot product	Yes	Medium
ColBERT	Token-level dense	MaxSim	Yes (tokens)	High
Cross-encoder	None (joint)	Full cross-attention	No	Very High

The two-stage paradigm: Retrieve cheaply with bi-encoder/BM25; rerank accurately with cross-encoder. This is the dominant production architecture.

Key Equations Cheat-Sheet

$$\text{BM25}(q, d) = \sum_t \text{IDF}(t) \cdot \frac{f_{t,d}(k_1 + 1)}{f_{t,d} + k_1 \left(1 - b + b \cdot \frac{|d|}{\text{avgdl}}\right)}$$

$$\mathcal{L}_{\text{DPR}} = -\log \frac{e^{\mathbf{q}^\top \mathbf{d}^+}}{e^{\mathbf{q}^\top \mathbf{d}^+} + \sum_j e^{\mathbf{q}^\top \mathbf{d}_j^-}}$$

$$\text{ColBERT: score}(q, d) = \sum_i \max_j \mathbf{Q}_i \cdot \mathbf{D}_j^\top$$

$$\text{RRF: score}(d) = \sum_r \frac{1}{K + \text{rank}_r(d) + 1}$$

$$\text{NDCG@}k = \frac{\sum_{i=1}^k r_i / \log_2(i + 1)}{\text{IDCG@}k}$$

My RAG System: hindyros/RAG

System Summary

A production-grade Retrieval-Augmented Generation backend built with FastAPI. Implements hybrid retrieval (BM25 + dense cosine), HyDE query expansion, LLM-based reranking, intent detection, and hallucination checking. Zero external vector database dependencies — all math in NumPy. Supports OpenAI and Mistral AI as configurable providers.

Repository File Structure

```
RAG/
+-- app/
|   +-- config.py          # Pydantic settings (all knobs in .env)
|   +-- main.py            # FastAPI app factory + CORS middleware
|   +-- dependencies.py    # DI: vector store, query pipeline, llm client
|   +-- api/
|       | +-- schemas.py    # Request/response Pydantic models
|       | +-- routes/
|       |     +-- health.py  # GET /health
|       |     +-- documents.py # GET/DELETE /documents
|       |     +-- ingest.py  # POST /ingest (PDF upload)
|       |     +-- query.py   # POST /query (question answering)
|       |     +-- visualize.py # GET /visualize (PCA + k-means)
|   +-- ingestion/
|       | +-- chunker.py     # 3-pass semantic chunking
|       | +-- pipeline.py    # extract -> chunk -> embed -> store
|   +-- retrieval/
|       | +-- bm25.py        # BM25kapi from scratch in NumPy
|       | +-- cosine.py      # Dense cosine similarity search
|       | +-- hybrid.py      # Reciprocal Rank Fusion
|       | +-- reranker.py    # LLM-based pointwise reranking
|   +-- query/
|       | +-- pipeline.py    # Full query orchestration
|       | +-- hyde.py        # HyDE query expansion
|   +-- intent/
|       | +-- detector.py    # Conversational vs. knowledge-seeking
|   +-- llm/
|       | +-- client.py      # Async httpx wrapper (Mistral + OpenAI)
|   +-- store/
|       | +-- vector_store.py # In-memory NumPy + .npz/.json persistence
|   +-- hallucination/
|       | +-- checker.py     # Vectara NLI per-sentence scoring
+-- pyproject.toml
+-- .env.example
+-- DECISIONS.md
```

Ingestion Pipeline

Before any query runs, documents must be processed and indexed. All ingestion is triggered by POST `/ingest` and orchestrated by `app/ingestion/pipeline.py`.

```
PDF upload (POST /ingest)
|
[1] Validation: MIME type + magic bytes ("%PDF") + size <= 50 MB
|       app/api/routes/ingest.py
|
[2] Duplicate check by filename
```

```

|         app/store/vector_store.py
|
[3] PDF text extraction with font metadata
|         app/ingestion/pdf_extractor.py
|
[4] Three-pass semantic chunking (512 tokens, 64 overlap)
|         app/ingestion/chunker.py
|
[5] Batch embedding (32 chunks per API call)
|         app/llm/client.py --> embed()
|
[6] Store all three indices + persist atomically to disk
|         app/store/vector_store.py

```

Ingestion Step 1 — Validation

Script: `app/api/routes/ingest.py`

Reads the first 4 bytes of the upload to verify the PDF magic bytes (%PDF). Also checks the MIME type (`application/pdf`) and enforces a 50 MB size cap. Non-PDF files are rejected before any processing begins.

Ingestion Step 2 — Duplicate Detection

Script: `app/store/vector_store.py`

Checks whether a document with the same filename already exists in the store. Prevents double-ingestion without requiring a full re-index.

Ingestion Step 3 — PDF Extraction

Script: `app/ingestion/pdf_extractor.py`

Uses **PyMuPDF** to extract text blocks alongside font-size and font-weight metadata. Each block is tagged `is_header = True` if its font is larger or bolder than the body text — this flag drives the structural split in the chunker. Returns an `ExtractedDocument` containing a list of `TextBlock` objects (text, page number, `is_header` flag).

Ingestion Step 4 — Three-Pass Semantic Chunking

Script: `app/ingestion/chunker.py`

See *Decision 1* for the full explanation. Output: a list of `Chunk` objects, each carrying `text`, `source_file`, `page_number`, `chunk_index`, and `section_header` (the nearest preceding header — used verbatim in citations).

Ingestion Step 5 — Batch Embedding

Script: `app/llm/client.py` (`embed()`)

Chunks are embedded in batches of 32 (the `MAX_EMBED_BATCH_SIZE` limit) to stay within API rate limits.

- Mistral: calls `/v1/embeddings` with `mistral-embed` → 1024-dim float32 vectors.
- OpenAI: calls `/v1/embeddings` with `text-embedding-3-small` → 1536-dim float32 vectors.

Ingestion Step 6 — Store & Persist

Script: `app/store/vector_store.py`

`VectorStore` maintains three synchronized indices updated together atomically:

- **CosineIndex** — NumPy matrix (N, D), pre-normalised; used for dense search.
- **BM25** — tokenised corpus + IDF table + tf-cache; used for sparse search.
- **Metadata list** — parallel list of **ChunkMetadata** (source file, page, header, doc id).

Persisted as **embeddings.npz** (NumPy compressed) and **metadata.json** (chunk metadata + BM25 state). Loaded on startup so the index survives restarts.

Query Pipeline — Overview

```

Query (POST /query)
|
v
[1] Intent Detection          app/intent/detector.py
    -> CONVERSATIONAL --> direct LLM reply, stop
    -> KNOWLEDGE_SEEKING --> continue
|
v
[2] HyDE Transformation      app/query/hyde.py
    LLM generates hypothetical answer document (temp=0.7)
    Embed it --> 1024-dim vector
|
v
[3] Hybrid Retrieval         app/retrieval/hybrid.py
    Dense: cosine_search(hyde_embedding) --> top-10
    Sparse: bm25_search(hyde_text)      --> top-10 [both use HyDE]
    Fusion: RRF (K=60)                 --> fused top-10
|
v
[4] Optional document filter app/query/pipeline.py
|
v
[5] LLM Reranking            app/retrieval/reranker.py
    10 x LLM calls (score 0-10 each) --> sorted top-5
|
v
[6] Similarity Gate          app/query/pipeline.py
    max(cosine_score) < 0.35 ? --> refuse with "insufficient_evidence"
|
v
[7] Answer Generation        app/query/pipeline.py
    prompt template by sub-intent --> LLM call (temp=0.1)
|
v
[8] Hallucination Check      app/hallucination/checker.py
    Vectara NLI per sentence --> overall = min(scores)
|
v
QueryResponse: answer + citations + grounded + hallucination scores

```

Query Pipeline — Step by Step

Step 1 — Intent Detection

Scripts: `app/intent/detector.py` · `app/llm/prompts/intent.py`

One LLM call (temperature=0.0, JSON mode) classifies the query on two axes:

- **Primary:** **CONVERSATIONAL** or **KNOWLEDGE_SEEKING** — determines whether retrieval runs at all.

- **Sub-intent:** FACTUAL, LIST, COMPARISON, or TABLE — selects the answer prompt template later.

If confidence < 0.60 , sub-intent is forced to FACTUAL but primary is kept. On any JSON parse failure the whole result defaults to `knowledge_seeking` / `factual` — erring on the side of doing retrieval (a wasted API call is cheaper than a missing answer).

Step 2 — Conversational Short-Circuit

Scripts: `app/query/pipeline.py (_handle_conversational()) · app/llm/prompts/conversational.py`

If primary = CONVERSATIONAL, the pipeline exits immediately. A direct LLM reply is generated (temperature = 0.5, max_tokens = 300) and returned. No embedding call, no BM25 scoring, no retrieval. The rest of the pipeline only executes for KNOWLEDGE_SEEKING queries.

Step 3 — HyDE Transformation

Scripts: `app/query/hyde.py · app/llm/prompts/hyde.py`

The LLM generates a hypothetical document that would plausibly answer the query (temperature = 0.7, max_tokens = 300). Temperature 0.7 adds diversity — a varied hypothetical covers a wider region of embedding space, which improves recall. The hypothetical document is then processed in two ways:

1. **Embedded** via `client.embed()` → 1024-dim float32 vector (used as the dense search vector in Step 4a).
2. **Kept as raw text** → tokenised by BM25 in Step 4b.

Both retrievers therefore search on the same information signal, making the RRF fusion in Step 4c more coherent.

Step 4a — Dense Cosine Retrieval

Script: `app/retrieval/cosine.py`

`CosineIndex.get_top_k()` normalises the HyDE embedding to a unit vector, then performs a single matrix-vector multiply against the pre-normalised corpus matrix:

$$\text{scores} = \mathbf{U} \hat{\mathbf{q}}, \quad \mathbf{U} \in \mathbb{R}^{N \times D}, \quad \hat{\mathbf{q}} \in \mathbb{R}^D$$

`np.argpartition` selects the top- k indices in $O(N)$ without a full sort. Returns `[(chunk_index, cosine_score), ...]`.

Step 4b — Sparse BM25 Retrieval

Script: `app/retrieval/bm25.py`

`BM25.get_top_k()` tokenises the HyDE document text, iterates over each query term, looks up its precomputed IDF, and accumulates BM25 scores across the full corpus using the cached per-document tf-counters:

$$\text{score}(d) += \text{IDF}(t) \cdot \frac{f_{t,d}(k_1 + 1)}{f_{t,d} + k_1(1 - b + b|d|/\text{avgdl})}, \quad k_1 = 1.5, \quad b = 0.75$$

Returns `[(chunk_index, bm25_score), ...]`.

Step 4c — RRF Fusion

Script: `app/retrieval/hybrid.py (reciprocal_rank_fusion())`

Both ranked lists are fed into `reciprocal_rank_fusion()`. For every document appearing in either list:

$$\text{rrf_score}(d) += \frac{1}{60 + \text{rank}(d) + 1}$$

The raw cosine and BM25 scores are **discarded** — only rank positions matter, making the fusion scale-free. The cosine score is **preserved separately** in the result dict because the similarity gate in Step 6 needs a real similarity value, not an RRF score.

Step 5 — Document Filter (optional)

Script: `app/query/pipeline.py`

If the POST `/query` request includes a `document_ids` list, the fused candidate list is filtered to only keep chunks whose `document_id` is in the allowed set. Enables scoped queries: “search only in this specific PDF.” Implemented as a list comprehension after RRF fusion, before reranking.

Step 6 — LLM Reranking

Scripts: `app/retrieval/reranker.py · app/llm/prompts/rerank.py`

The top-10 RRF candidates each get one LLM call (`temperature=0.0`, `max_tokens=150`, JSON mode). The LLM sees the **original user query** (not the HyDE document) + the candidate chunk text, and returns:

```
{"score": 7, "reasoning": "Directly addresses the mechanism of..."}
```

Score is clamped to `[0, 10]`. Candidates sorted by (`rerank_score` DESC, `cosine_score` DESC) — cosine breaks ties. Top-5 are kept. On any failure: `score=5` (neutral), candidate stays in pool.

Step 7 — Similarity Gate

Script: `app/query/pipeline.py`

After reranking, the pipeline checks the **maximum** cosine score across the reranked top-5 (not just position 0):

```
best_cosine = max(r["cosine_score"] for r in reranked)
if best_cosine < settings.min_similarity_threshold: # default 0.35
    return refusal("insufficient_evidence")
```

Using `max()` rather than position-0 guards against a case where reranking promoted a BM25-heavy chunk (high BM25, low cosine) to the top. A confident refusal is returned rather than risking a hallucinated answer.

Step 8 — Answer Generation

Scripts: `app/query/pipeline.py · app/llm/prompts/answer/{factual, list_format, comparison, table}.py`

The sub-intent from Step 1 selects the prompt template:

Sub-intent	Prompt script
FACTUAL	<code>app/llm/prompts/answer/factual.py</code>
LIST	<code>app/llm/prompts/answer/list_format.py</code>
COMPARISON	<code>app/llm/prompts/answer/comparison.py</code>
TABLE	<code>app/llm/prompts/answer/table.py</code>

Chunks are passed as numbered citations `[1], [2], ...` LLM call: `temperature=0.1` (near-deterministic for factual reliability), `max_tokens=1500`.

Step 9 — Hallucination Check

Script: `app/hallucination/checker.py`

Runs inside `asyncio.to_thread()` to avoid blocking the async event loop. The Vectara NLI model scores each sentence of the generated answer for consistency with the retrieved context:

$$\text{overall_score} = \min_{s \in \text{sentences}} \text{NLI}(\text{context}, s)$$

Sentences with score < 0.5 are flagged as potentially hallucinated. The minimum (not average) is deliberate: one hallucinated sentence invalidates the whole answer regardless of how grounded the rest is. The result is attached to `QueryResponse.hallucination` — it never blocks the response, it is metadata for the caller to inspect or surface in a UI.

Architectural Decisions & Justifications

Decision 1: Three-Pass Semantic Chunking (512 tokens, 64 overlap)

What:

1. **Pass 1 — Structure:** PyMuPDF span-level font metadata detects headers; chunk boundaries placed at section breaks, never mid-header.
2. **Pass 2 — Sentences:** Accumulate sentences to 512-token target; deliberately overshoot rather than cut mid-sentence.
3. **Pass 3 — Overlap:** Add 64-token overlap ($\sim 12.5\%$) between adjacent chunks to handle answers spanning chunk boundaries.

Why 512 tokens? Empirical sweet spot across IR benchmarks — large enough for context, small enough for precise embedding.

Why PyMuPDF over pdfplumber? Font metadata enables header detection. pdfplumber requires fragile regex heuristics for the same task.

Why overlap? If an answer spans two chunks, overlap ensures at least one chunk contains the full relevant span.

Decision 2: In-Memory NumPy Store (No External Vector DB)

What: All embeddings stored as NumPy arrays in memory, persisted atomically to `.npz` + `.json` files in `./data/store`.

Why not Postgres/pgvector or FAISS?

- Zero operational overhead — no external services to deploy and maintain.
- Full transparency: every line of the retrieval math is visible and testable.
- Performance is adequate: sub-100ms query latency on CPU up to $\sim 1\text{M}$ chunks.
- At $> 1\text{M}$ chunks ($\sim 4\text{GB}$ RAM) a dedicated vector DB would be warranted.

Performance detail:

- Pre-normalizes embeddings for $O(N \times D)$ cosine similarity.
- Uses `np.argpartition` for $O(N)$ top-k selection (not $O(N \log N)$ sort).
- Caches normalized matrix when corpus is unchanged.

Decision 3: Hybrid BM25 + Dense Cosine via RRF

What: Two independent first-stage retrievers fused by Reciprocal Rank Fusion:

$$\text{RRF-score}(d) = \frac{1}{60 + \text{rank}_{\text{cosine}}(d) + 1} + \frac{1}{60 + \text{rank}_{\text{BM25}}(d) + 1}$$

Why hybrid?

- BM25 excels at exact keyword matches (named entities, technical terms, rare words).
- Dense cosine excels at semantic similarity (synonyms, paraphrases, conceptual queries).
- Hybrid consistently outperforms either alone by 3–8 points on standard benchmarks.

Why RRF instead of linear combination?

- Rank-based: no parameter tuning for score scale normalization.
- BM25 scores (unbounded positive floats) and cosine scores ($[0,1]$) are incompatible scales.
- RRF is scale-free — only ranks matter.

Why custom BM25 in NumPy? Transparency + no external search library dependency. BM25 is trivial to implement; a library adds complexity with no benefit at this scale.

Decision 4: HyDE Query Expansion

What: LLM generates a hypothetical document that would answer the query → embedded → used for dense search. Original query still used for BM25.

Why?

- Queries (“when did X happen?”) and documents (“X occurred on Y..”) have different linguistic distributions; they inhabit different regions of embedding space.
- HyDE generates a document-like embedding from the query side, unifying the dense search signal.
- Empirically improves dense retrieval recall on knowledge-dense corpora.

Cost: One extra LLM call per query (temperature 0.7, max 300 tokens).

Correction — both retrievers use the HyDE document: The hypothetical document is fed to *both* retrievers, not just the dense one. Its embedding goes to cosine search; its raw text is tokenised by BM25. The code comment explains why: if cosine used the HyDE embedding but BM25 used the original query, the two retrievers would be searching for *different* things, weakening the RRF fusion assumption. The hypothetical document is a natural keyword expansion of the query — written in document style, using the vocabulary the corpus uses — so BM25 benefits from it too. Both retrievers now represent the same information need.

Decision 5: LLM-Based Pointwise Reranking

What: Each of the top-10 RRF candidates is scored 0–10 by the LLM with reasoning. Candidates re-sorted by rerank score; top-5 kept.

Why LLM reranking?

- Applies question-specific cross-attention (what makes a passage relevant to *this* question, not just any question).
- Counters the “lost in the middle” bias — reranking puts the best passage first.
- Handles comparative / list / table queries that require a holistic view of relevance.

Tradeoff: ~0.3s per candidate, ~3s for 10 candidates. **Production note:** The code explicitly acknowledges that a cross-encoder model (e.g., `cross-encoder/ms-marco-MiniLM-L-6-v2`) would reduce this to ~50ms. LLM reranking was chosen here for implementation simplicity.

Fallback: If scoring fails (malformed output), uses RRF score rescaled to 0–10.

Decision 6: Intent Detection with JSON-Mode Classification

What: LLM classifies each query on two axes using Mistral JSON mode:

1. **Routing:** Conversational → skip retrieval; Knowledge-seeking → full pipeline.
2. **Format:** Factual / List / Comparison / Table (selects prompt template).

Why? Avoids wasting API calls on “Hi, how are you?” type queries. Also selects specialized prompt templates that improve answer quality.

Fallback strategy: Defaults to knowledge-seeking on any parse failure. Rationale: false negative (missed conversational) = one wasted API call; false positive (missed knowledge-seeking) = no answer. Asymmetric cost favors defaulting to retrieval.

Confidence threshold: 0.60 — below this, defaults to knowledge-seeking.

Decision 7: Similarity Threshold Gate (0.35 cosine)

What: After reranking, if the top chunk's cosine similarity < 0.35 , the system refuses to answer, returning `"insufficient_evidence"`.

Why? Prevents hallucination on out-of-scope questions. If the corpus doesn't contain relevant information, a forced generation is likely to hallucinate. A confident refusal is more useful than a confident wrong answer.

Tuning: 0.35 is a conservative default. Lower $\rightarrow$ more answers, more hallucination risk; higher $\rightarrow$ more refusals, higher precision.

Decision 8: Vectara NLI Hallucination Checker

What: After generation, each sentence of the answer is scored for consistency with the retrieved context using the Vectara NLI model:

$$\text{overall_score} = \min_{\text{sentence } s} \text{NLI}(\text{context}, s)$$

Sentences with score < 0.5 are flagged as potentially hallucinated.

Why minimum (not average)? Chain-as-strong-as-weakest-link: one hallucinated sentence invalidates the answer regardless of how grounded the rest is.

Why Vectara NLI? Lightweight, open-source, no API call needed. Runs locally with automatic CUDA detection. Designed specifically for hallucination detection.

Decision 9: Async httpx Client (Not Provider SDKs)

What: LLM API calls use a custom `httpx.AsyncClient` wrapper with connection pooling and retry logic, rather than official Mistral/OpenAI SDKs.

Why?

- Avoid SDK version drift — provider SDKs change frequently and can break silently.
- Full control over retry behavior (max 3 retries, 1.5x exponential backoff, retry on 429/5xx, never retry 4xx).
- `httpx` supports both sync and async with identical interfaces.

Configuration Reference

Parameter	Default	Effect
LLM_PROVIDER	mistral	mistral or openai
CHUNK_SIZE_TOKENS	512	Target chunk size in tokens
CHUNK_OVERLAP_TOKENS	64	Overlap between adjacent chunks
RETRIEVE_TOP_K	10	Initial hybrid retrieval depth
RERANK_TOP_K	5	Final results after reranking
RRF_K	60	RRF constant (higher = gentler rank penalty)
SIMILARITY_THRESHOLD	0.35	Min cosine score to attempt answer
MIN_INTENT_CONFIDENCE	0.60	Intent classification threshold
MAX_EMBED_BATCH_SIZE	32	Embeddings per API call
VECTOR_STORE_PATH	./data/store	Persistence directory

Provider Switching Warning

Switching LLM_PROVIDER requires deleting `./data/store` and re-ingesting all documents. OpenAI embeddings are 1536-dim; Mistral embeddings are 1024-dim. The stored `.npz` matrix dimensions are incompatible between providers.

Embedding Models

Provider	Embedding Model	Dimensions	Chat Model
Mistral	mistral-embed	1024	mistral-large-latest
OpenAI	text-embedding-3-small	1536	gpt-4o-mini

API Endpoints

Method	Path	Purpose
GET	/health	Status, chunk count, document count
GET	/documents	List all ingested documents
DELETE	/documents/{id}	Remove a document and all its chunks
POST	/ingest	Upload PDFs (multipart, max 50MB)
POST	/query	Question answering with citations
GET	/visualize	2D PCA + k-means cluster visualization

Performance Characteristics

Operation	Typical Latency
Dense cosine search (500K chunks, CPU)	< 100ms
BM25 sparse search	< 50ms
Intent detection	~200ms (1 LLM call)
HyDE expansion	~500ms (1 LLM call)
LLM reranking (10 candidates)	~3s (10 LLM calls)
Answer generation	~1–2s
Hallucination checking	~500ms (local model, CPU/GPU)
Total end-to-end	~5–7s per query
Scaling boundary	~1M chunks (~4GB RAM)

Key Interview Talking Points

Questions You Should Be Ready to Answer

- Why not just use a vector database like Pinecone or Weaviate?**
At this scale (<1M chunks) the operational overhead of running and maintaining a dedicated vector DB isn't justified. NumPy gives us full transparency — I can debug every retrieval operation. At 1M+ chunks I'd migrate to Postgres + pgvector or FAISS.
- Why hybrid retrieval? Isn't dense search enough?**
Dense retrieval struggles with exact matches (named entities, rare technical terms, product names). BM25 handles these well but misses semantic similarity. The two are complementary, and RRF fusion reliably outperforms either alone by 3–8 points.
- What is HyDE and why did you use it?**
A query like "What year did X happen?" lives in a different embedding space than the answer document. HyDE asks the LLM to generate a hypothetical answer, then embeds *that* for dense search. It bridges the query-document distributional gap at the cost of one extra LLM call.
- Why does your reranker make N LLM calls instead of using a cross-encoder?**
Implementation simplicity for this prototype. In production I'd replace with a cross-encoder model (ms-marco-MiniLM-L-6-v2) which would cut reranking from ~3s to ~50ms.

5. What does the similarity threshold gate do?

Before generating an answer, I check whether the top retrieved chunk has cosine similarity ≥ 0.35 . If not, I return “insufficient_evidence” rather than generating a likely-hallucinated answer. A confident refusal is more valuable than a confident wrong answer.

6. How does your hallucination checker work?

After generation, I run each sentence of the answer through Vectara’s NLI model against the retrieved context. Any sentence with consistency score < 0.5 is flagged. The overall score is the minimum sentence score — one hallucinated sentence fails the whole answer.

7. Why 512 tokens and 64 overlap for chunking?

512 is the empirical sweet spot from IR benchmarks — large enough to provide context, small enough for precise embeddings. 64 tokens ($\sim 12.5\%$ overlap) ensures that answers spanning two chunks are still fully contained in at least one.

8. How does intent detection work and why is the fallback knowledge-seeking?

The LLM classifies queries as conversational (no retrieval needed) or knowledge-seeking (full pipeline). Fallback is knowledge-seeking because the cost asymmetry is clear: a false positive means one wasted API call; a false negative means no answer at all.

9. What would you change to scale this to 10M documents?

(1) Replace NumPy store with Postgres + pgvector or FAISS for ANN search. (2) Replace LLM reranking with a cross-encoder model. (3) Add a caching layer (Redis) for repeated queries. (4) Add async ingestion queue (Celery/RQ) for large batch uploads.